# AchiFL: Efficient Asynchronous Hierarchical Federated Learning via Adaptive Edge Aggregation

Xiangyun Tang, *Member, IEEE*, Ruiqi Peng, Hao Li, Tao Zhang, Xiangwang Hou, *Member, IEEE*, and Xiangzhi Liu

***Abstract*— The rapid growth of uncrewed aerial vehicle (UAV) has increased the demand for secure and efficient training of Artificial General Intelligence models. The Low Altitude Economy (LAE), including precision agriculture, urban mobility, and logistics, further emphasizes the need for efficient machine learning frameworks due to the large volumes of heterogeneous data generated by UAV and constraints in communication bandwidth and energy. In response, Federated Learning (FL) has emerged as a promising solution, offering privacy-preserving capabilities while enabling collaborative model training across distributed devices. This paper introduces an efficient Asynchronous Hierarchical Federated Learning (AHFL) framework, called `AchiFL`, which supports asynchronous model uploads by dynamically adjusting the clock and clusters clients with similar data distributions for hierarchical aggregation. Although the existing AHFL architecture mitigates prolonged server wait times through asynchronous clocks and alleviates communication bottlenecks via hierarchical aggregation, it remains susceptible to performance degradation due to convergence delays and data heterogeneity at the edge layers. To tackle these challenges, `AchiFL` incorporates a two-phase optimization approach. First, a clustering-based gradient pruning mechanism groups clients with similar data distributions under edge servers and resolves gradient conflicts using directional pruning and amplification, thereby accelerating convergence under non-Independent and Identically Distributed (non-IID) conditions. Furthermore, through rigorous proof, we identify the key metrics influencing model performance and convergence. By integrating model obsolescence, prediction accuracy, and client participation, we quantify the obsolescence effect at the edge servers and establish a dynamic edge weight penalty mechanism. This mechanism enables adaptive re-weighting of the edge models during the global aggregation process. Extensive experiments show that `AchiFL` achieves an accuracy improvement of 3.10% to 13.23% on non-IID datasets.**

***Index Terms*— Asynchronous hierarchical federated learning, uncrewed aerial vehicle (UAV), clustering strategy, gradient pruning and growth, dynamic weighting.**

## I. Introduction

RECENT advances in the Internet of Things, Artificial General Intelligence (AGI) and communication technologies have fostered the development of edge Artificial Intelligence (AI), where intelligence is pushed from the core network to the edge by deploying AI models on the user equipment. The rapid growth of the Low Altitude Economy (LAE), encompassing uncrewed aerial vehicle (UAV)-enabled services such as applications spanning urban mobility [1], [2], precision logistics [3], [4], smart agriculture [5], [6], [7], and intelligent tourism systems [8], [9], have introduced new challenges in model training and data processing. In this context, UAV are expected to handle sensitive tasks while operating in resource-constrained environments, where communication efficiency and data privacy are paramount. These autonomous systems heavily rely on multi-sensor data fusion, where heterogeneous inputs from visual, spatial, and operational sensors are integrated to enhance decision-making robustness [10]. Although UAV with sensors generate vast datasets crucial for AGI model enhancement, centralized training paradigms require raw data transmission to cloud servers. This practice creates significant vulnerabilities in privacy protection, especially when dealing with sensitive geospatial or operational data. To address these concerns, Federated Learning (FL) [11], [12] has emerged as a privacy-preserving alternative. FL enables localized model training on client devices and restricts data sharing to gradient updates, effectively decoupling model improvement from raw data exposure. However, deploying FL in UAV networks brings unique challenges that require systematic resolution.

The increasing demand for real-time data processing in LAE requires efficient FL techniques that minimize communication overhead and adapt to varying network conditions. The deployment of FL in UAV networks faces some challenges: resource-constrained edge devices, statistical heterogeneity in distributed datasets, and increasing communication overhead. These factors degrade training efficiency, model generalization, and operational sustainability, as demonstrated through

Received 26 May 2025; revised 16 September 2025 and 18 October 2025; accepted 30 November 2025. Date of publication 8 December 2025; date of current version 13 January 2026. This work was supported in part by the National Nature Science Foundation of China under Grant 62572502, Grant 62402029, and Grant 62302539; in part by the National Natural Science Foundation of China under Grant 623B2060; in part by the Hainan Provincial Natural Science Foundation Lingshui Li'an International Education Innovation Experimental Zone Joint Project under Grant 624LALH005; in part by the Key Laboratory of Computing Power Network and Information Security, Ministry of Education under Grant 2024ZD014; in part by the National Cryptologic Science Fund of China under Grant 2025NCSF02030; and in part by the Beijing Natural Science Foundation under Grant L251041. The associate editor coordinating the review of this article and approving it for publication was J. Wang. *(Corresponding author: Xiangyun Tang.)*

Xiangyun Tang, Ruiqi Peng, and Hao Li are with the Key Laboratory of Ethnic Language Intelligent Analysis and Security Governance of MOE and the School of Information Engineering, Minzu University of China, Beijing 100081, China (e-mail: xiangyunt@muc.edu.cn; ruiqi_peng@muc.edu.cn; 24302177@muc.edu.cn).

Tao Zhang is with the School of Cyberspace Science and Technology, Beijing Jiaotong University, Beijing 100044, China (e-mail: taozh@bjtu.edu.cn).

Xiangwang Hou is with the Department of Electronic Engineering, Tsinghua University, Beijing 100084, China (e-mail: xiangwanghou@163.com).

Xiangzhi Liu is with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan 250316, China (e-mail: liuxzh@sdas.org).

This article has supplementary downloadable material available at https://doi.org/10.1109/TCCN.2025.3641522, provided by the authors.

Digital Object Identifier 10.1109/TCCN.2025.3641522

2332-7731 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.

empirical analyses [13], [14], [15], [16]. The limited computational resources of UAV edge devices, combined with traditional synchronization protocols that wait for all nodes to complete local updates, extend training times and exacerbate delays [17]. Additionally, frequent model updates and large data transfers in resource-constrained environments, where UAV have limited bandwidth, lead to high communication costs, hindering timely model aggregation and reducing training effectiveness [18]. Moreover, UAV' geographically constrained operational domains and environmental variations lead to significant data heterogeneity, worsening the issue of non-Independent and Identically Distributed (non-IID) data [19]. This issue drives local models toward client-specific optima, impeding global convergence and degrading cross-domain generalization [20]. These combined challenges of computational asynchrony, data heterogeneity, and communication bottlenecks hinder the effective application of FL in LAE.

We design Asynchronous Hierarchical Federated Learning (AHFL) to overcome the persistent challenges of device heterogeneity and communication bottlenecks in FL. This framework builds upon established asynchronous coordination mechanisms [21] and multi-tier aggregation architectures [22]. **(i)** The asynchronous mechanism reduces the waiting time on the server side, allowing the server to aggregate the received local models within a dynamic clock cycle without needing to wait for all clients (e.g., UAV devices) to complete their local model training [23], [24]. However, this implies that slower clients, characterized by longer local computation times, may fail to contribute their local data to the global model promptly. This delay exacerbates the discrepancy between their local and global models, leading to the obsolescence effect [25], [26]. **(ii)** The hierarchical mechanism in FL adopts a two-tier architecture, where a group of similar clients submits their local models to an edge server (a UAV device with more substantial computational power). The edge server aggregates these into an edge model and forwards them to the central server (e.g., a base station) for global model updates [22], [27]. By routing updates through the edge server, Hierarchical Federated Learning (HFL) reduces communication frequency and volume, thus mitigating network congestion and delays. However, aggregating similar client data by the edge server may exacerbate the non-IID data issues, hindering global convergence and generalization across different data distributions.

As discussed above, existing AHFL struggles to effectively mitigate the issues arising from the non-IID nature of data distribution while simultaneously introducing the obsolescence effect, thereby compromising the global model's convergence speed and generalization capabilities. **(i)** One method to mitigate the non-IID data issues in existing methods is to have the server distribute uniform data to each client, thereby reducing data differences between clients [28]. However, in the complex AHFL scenario, if the central server distributes the data, it may not have access to a complete dataset. Suppose each edge server distributes data to its clients. In that case, it cannot ensure data disparity between edge servers. Maintaining a protocol for balanced data distribution across all edge servers is challenging. Another method involves the server sampling data from each client and then adjusting the data to make the distribution uniform across all clients [29]. With numerous clients, sampling from each client is undoubtedly tricky. Additionally, some clients may be unwilling to share data due to privacy concerns, and client responses to sampling requests can lead to prolonged server waiting times. Furthermore, both methods alter client data, inevitably causing a deviation between the aggregated global model and the original global model. **(ii)** Existing methods for addressing the obsolescence effect focus on its impact on servers in Asynchronous Federated Learning (AFL) due to infrequent local model updates [26], [30]. However, in AHFL, the aggregation of local models by edge servers masks the obsolescence effect generated by clients. Applying existing methods to each edge server is challenging because the degree of obsolescence effect among clients under different edge servers varies, making it difficult to uniformly handle the obsolescence effect across edge servers, thus leading to discrepancies without effectively addressing the issue. Solving the obsolescence effect in AHFL involves quantifying this metric. There are insufficient mechanisms to dynamically adjust and balance the impact of the obsolescence effect from different edge models on global model aggregation from the edge server to the central server phase.

In this paper, we propose an efficient Asynchronous Hierarchical Federated Learning, called `AchiFL`, which mitigates the issues of non-IID data and the obsolescence effect to increase the convergence speed of the global model and enhance its generalization capabilities. `AchiFL` consists of a clustering pruning mechanism to mitigate the issues of non-IID data and increase the convergence speed and a weight penalty mechanism to address the obsolescence effect.

Mitigating the non-IID data issues by adjusting the client data is undoubtedly tricky in AHFL. Therefore, we propose a clustering pruning mechanism which alleviates this issue from the perspective of gradients. First, we employ a clustering algorithm to cluster the clients with similar data distribution under the same edge server so that the local models do not differ much. After that, the edge servers perform gradient pruning for those whose gradient direction is opposite to the global model and gradient growth for those whose gradient direction is the same as the global model by checking the gradient between different clients, which can induce the aggregated edge model to be consistent with the gradient of the global model, thus improving the convergence speed.

Since edge servers aggregating local models mask the obsolescence effect generated by clients in AHFL, we propose a weight penalty mechanism in the edge server to the central server phase to address this issue. The mechanism quantifies the metric that edge servers are affected by the local model's obsolescence effect and dynamically adjusts and balances the edge model by considering its accuracy and the number of aggregated clients.

The main contributions are summarized as follows:

- We propose an innovative asynchronous hierarchical federated learning framework that facilitates model updates asynchronously by dynamically adjusting the global clock and cluster clients with similar data distributions for hierarchical aggregation.
- We propose an innovative pruning and growth strategy that mitigates the non-IID issue by leveraging

Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.

gradient-based methods to process the locally uploaded models from each client.
- Through a comprehensive analysis of the factors influencing global model performance, we design a dynamic weighting strategy that resolves the obsolescence effect by adapting the weights of various metrics in real-time.
- Experimental results show that our proposal `AchiFL` improves accuracy by 3.10% to 13.23% on non-IID data and effectively reduces time cost overhead compared to state-of-the-art methods.

## II. Related Work

The rapid rise of the LAE and semantic-aware networking is reshaping how edge intelligence is provisioned in aerial and vehicular systems, generative-AI enabled spectrum cartography, and robust LAE networking highlight the need to couple learning with dynamic radio resources [31], [32], while semantic-aware radio access network architectures formalize task-level metrics beyond bit-level fidelity [33], [34]. Recent overviews further consolidate the opportunities and system challenges across ground-to-sky wireless stacks [35] and secure sensing and integrated sensing and communication pipelines [36], alongside evidence that evolutionary multi-objective control is effective for reliable aerial collaboration [37]. These trends motivate AHFL for UAV, where timeliness, heterogeneity, and cross-tier coordination are first-class concerns. This section provides a comprehensive overview of the significant research related to the application of FL in UAV, focusing on AFL to address long server waiting times, HFL to reduce communication overhead, and design strategies for AHFL. The challenges faced by different methods under various mechanisms are summarized in Table I.

### A. Federated Learning

The security surface of AI-driven perception and navigation continues to expand, perturbation-aided model and encoder stealing [38] and adversarial object fusion against vision-language navigation [39] illustrate realistic risks in mobile sensing and services. These observations reinforce our choice to design FL updates and aggregations that remain stable under distribution shifts and potentially adversarial operating conditions. FL achieves decentralized model training but faces challenges like prolonged server waiting times and high communication overhead. When clients complete local training at different times, the aggregation process gets delayed, reducing system efficiency. Researchers explore strategies to mitigate waiting times, such as adaptive client selection [40], to ensure faster model updates. Communication overhead is also a significant bottleneck, particularly in bandwidth-limited environments, as frequent data exchanges are required for model updates. Techniques like FedAvg [12], which reduces the frequency of communication by allowing clients to train for multiple iterations before sending updates, and QSGD [41], which compresses model updates to reduce communication costs, are proposed to address this issue. Despite these advancements, balancing communication efficiency with model performance remains a key area of research in FL systems.

FL improves decentralized data processing, enhances system intelligence, ensures data privacy, and reduces communication burdens across various applications, including UAV. Zhang and Hanzo [40] propose a method that integrates FL with Weighted Zero-Forcing transmission precoding based on imperfect Channel State Information, interacting with a central system to achieve high classification accuracy while minimizing communication costs. Pham et al. [42] develop an algorithm that uses decomposition techniques and convex approximation to optimize multiple parameters such as transmission time, bandwidth allocation, power control, and UAV placement, thus enhancing transmission efficiency in FL-based systems. Hou et al. [43] propose a strategy using distributed proximal policy optimization to optimize UAV trajectory, transmission power, and resource allocation, which also improves local model accuracy within the FL framework. These studies [44], [45], [46], [47] demonstrate how FL is leveraged to optimize resource usage and communication efficiency, ensuring privacy while reducing overhead in distributed environments. Despite progress, challenges persist in UAV-based FL systems, particularly prolonged server waiting times during client interactions, which slow down model aggregation and lead to inefficient use of time and resources. Furthermore, due to frequent data transmission between UAV and central or edge servers, high communication overhead becomes a bottleneck in resource-constrained environments with limited bandwidth. Effective strategies are needed to reduce server waiting times and communication costs and improve the performance of FL.

### B. Asynchronous Federated Learning

AFL reduces server-side waiting times but simultaneously introduces the obsolescence effect. Lu et al. [48] propose a privacy-preserving AFL mechanism for edge network computing, allowing nodes to join or leave the learning process anytime. Nguyen et al. [49] propose a novel buffered asynchronous aggregation method, FedBuff, compatible with differential privacy and secure aggregation. Several works [50], [51], [52], [53] propose multi-parameter dynamic selection model-based aggregation of clients to mitigate prolonged server waiting times. Although these works decrease the prolonged server waiting times, they overlook the impact of the obsolescence effect in asynchronous FL, which can undermine the performance and convergence of the global model.

The improvement of AFL in UAV applications has mainly focused on device selection, resource management, and optimizing data value and scheduling, aiming to enhance learning efficiency and model accuracy. Zhai et al. [54] propose a wireless AFL where UAV function as parameter servers, facilitating wireless data collection and continuous model updates, thus significantly boosting learning efficiency. Yang et al. [46] introduce a device selection strategy in their AFL to prevent low-quality devices from impacting learning efficiency and accuracy. They also develop a joint device selection, UAV

Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.

placement, and resource management algorithm based on the asynchronous advantage actor-critic method, which enhances FL convergence speed and accuracy. Cui et al. [55] propose a two-stage AFL scheme for UAV. In the pre-training phase, they model the learning process as a cooperative game with monotonicity and submodularity. They use shapley values to quantify UAV data value and derive an upper bound for estimation error. During the training phase, they introduce the Age of Update (AoU) to address fairness and use the whittle index method to minimize AoU and optimize UAV scheduling. While significant progress has been made in reducing server waiting times in AFL for UAV, existing solutions often introduce the obsolescence effect, which undermines convergence. Zhai et al. [54], Yang et al. [46], and Cui et al. [55] address waiting times and fairness. However, their solutions do not fully mitigate the negative effects of outdated updates, which pose challenges in practical UAV applications.

### C. Hierarchical Federated Learning

While HFL reduces costly communication overhead, it exacerbates the non-IID data issues. Liu et al. [22] propose a client-edge-cloud HFL system supported by the HFed algorithm, which allows multiple edge servers to perform partial model aggregation. This approach enables faster model training and achieves a better trade-off between communication and computation. Abad et al. [56] propose federated edge learning, where mobile users collaborate by sharing model parameters instead of datasets, assisted by a mobile base station. They optimize resource allocation to reduce communication delays, introduce small cell base stations to coordinate UAV, and exchange model updates with the server for global consensus. Briggs et al. [27] modify FL by introducing a hierarchical clustering step, which separates client clusters based on the similarity between local updates and the global model. Once separated, clusters independently train on dedicated models, improving training for non-IID data. Although these approaches diminish communication overhead and improve efficiency, they fail to sufficiently address the challenges posed by non-IID data, which continues to hinder model convergence and generalization across heterogeneous clients.

The applications of HFL in UAV aim to improve model convergence and data transmission efficiency by addressing UAV-client matching, time allocation, energy consumption, model performance, UAV placement, and user selection strategies. Wang et al. [57] propose a two-tier HFL scheme which UAV offload data to base stations and relay models between clients and servers. They optimize data convergence and transmission using an efficient algorithm that combines subgradient methods and a genetic algorithm based on cross-entropy. Wang et al. [58] introduce a hierarchical nested personalized FL method, optimizing both individual UAV and swarm-level systems for energy consumption and model performance. Khelf et al. [59] propose a UAV-assisted HFL with optimized UAV placement, user-UAV associations, and channel allocation to minimize training time. Although these methods reduce communication overhead, they do not fully address the challenges posed by non-IID data, which remains a significant barrier to effective HFL performance.

### D. Asynchronous Hierarchical Federated Learning

Beyond learning algorithms, system-level coordination in mobile aerial and vehicular networks has advanced via multi-layer consensus and consortium-chain mechanisms for cross-domain data sharing [60]. Such designs complement modern LAE roadmaps [35] and robust aerial collaboration protocols [37], providing practical substrates on which AHFL can operate under intermittent links and tiered aggregation.

AHFL struggles to mitigate the issues arising from the non-IID nature of data distribution while simultaneously introducing the obsolescence effect, thereby compromising the global model's convergence speed and generalization capabilities. Miao et al. [61] propose an AHFL that divides data request tasks into multiple subtasks and utilizes deep reinforcement learning methods to select data nodes for model training. Yang et al. [62] propose an efficient AHFL that combines gradient sparsification and asynchronous aggregation techniques. Huff et al. [63] propose a decentralized AHFL, which enhances system scalability and robustness by adopting a decentralized layered network structure and a two-phase model aggregation paradigm. Su et al. [64] propose KNOT, which divides clients into multiple clusters, with aggregation only occurring within each cluster. They also formulated a dictionary minimization issue to optimize client cluster allocation. Yang et al. [65] propose an AHFL based on bandwidth allocation and client scheduling, which dynamically allocates the edge server based on the client's mobility during training, considering the edge server's remaining bandwidth. Currently, the application of AHFL in UAV systems has not yet fully addressed the critical challenges of the obsolescence effect and non-IID data, and strategies to mitigate these issues are urgently needed.

This paper proposes that `AchiFL` addresses the aforementioned challenges. Unlike previous approaches that address the non-IID issue from a data perspective, `AchiFL` seeks to mitigate this problem from the gradient perspective. Additionally, we identify the key factors influencing the accuracy and convergence of `AchiFL` through mathematical proof. By treating the obsolescence effect as both a weight indicator and a penalty term, `AchiFL` minimizes the impact of clients with significant obsolescence on the global model. Our adaptive edge aggregation and time-aware weighting resonate with cost-sensitive and imbalance-aware optimization principles [66], [67] and with graph and spectral perspectives that account for heterophily in relational data [68], offering a complementary lens for prioritizing timely and informative contributions in asynchronous tiers.

## III. Preliminaries & Problem Definition

In this section, we first introduce the typical AHFL and then discuss the challenges that motivate the definitions of key problems that lead to the design of `AchiFL`.

Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.

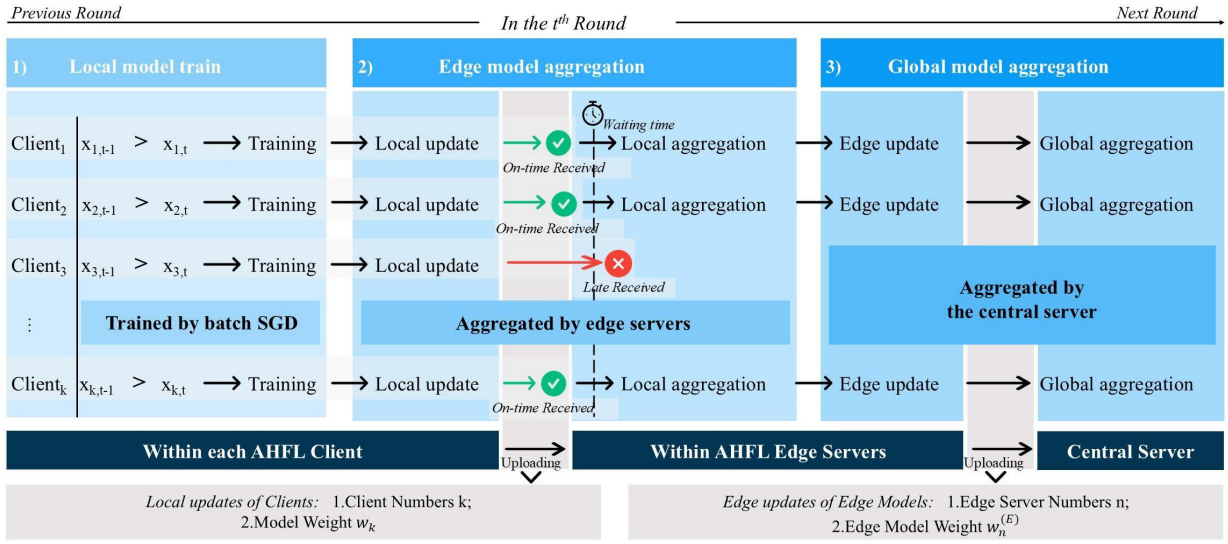



Fig. 1. The workflow of the typical asynchronous hierarchical federated learning framework.

TABLE I
CHALLENGES OF DIFFERENT METHODS

| Method | Challenge | | | |
|---|---|---|---|---|
| | (1) | (2) | (3) | (4) |
| FL | × | × | × | × |
| AFL | ✓ | × | × | × |
| AHFL | ✓ | ✓ | × | × |
| Ours AchiFL | ✓ | ✓ | ✓ | ✓ |

* Where ✓ denotes that the method addresses one of the four challenges—(1) prolonged server wait times, (2) costly communication overhead, (3) the obsolescence effect, and (4) non-IID data issues—while × indicates that the method does not resolve the respective challenge.

### A. Asynchronous Hierarchical Federated Learning

We introduce a general AHFL [63] method here. AHFL usually consists of three main components: clients (UAV devices), edge servers (UAV devices with more substantial computational power), and a central server (base station). Clients train local models and then asynchronously upload them to edge servers for edge model aggregation. The edge servers then asynchronously upload the aggregated edge models to the central server for global model aggregation. After completing one round of training, the aggregated global model is distributed to each client for the next round of training. This process continues until $t$ training rounds are completed, and the global model converges. The training process of AHFL is shown in Figure 1. The whole training process of AHFL can be divided into the following steps:

1) **Clients Train Local Model**. Each client begins its multi-round local batch SGD training task with an identical untrained model passed from the central server:

$$\omega_k = \omega_k - \eta_g \frac{1}{\beta} \sum_{x_k \in \mathcal{X}_k} \Delta f(\omega_k; \omega_k), \tag{1}$$

where $\mathcal{X}_k$ is a mini-batch of the training samples of client $k$, $\beta$ is the batch size, and $\eta_g$ is the global learning rate, which is used by each client for its local model updates.

2) **Edge Servers Aggregate Local Models**. Once training is complete, clients send their trained models back to their corresponding edge server. The edge server performs the weighted average on the received models to produce the updated edge model, where is $\lambda_{agg,k}$ the aggregation weight for client $k$:

$$\omega_n^{(E)} = \sum_{k=1}^{K} \lambda_{agg,k} \omega_k, \tag{2}$$

where $\omega_n^{(E)}$ is the updated edge server model, $K$ is the number of clients covered by the edge server, $\omega_k$ is the model weights of client $k$, and $\sum_{k=1}^{K} \lambda_{agg,k} = 1$.

3) **Global Server Aggregates Edge Models**. Once an edge server completes the aforementioned local model aggregation process, it sends the edge model to the central server. Upon receiving all edge models, the central server performs a weighted average of the received edge models to generate an updated global model, where $\lambda_{agg,n}$ is the weight of edge server $n$:

$$\omega^{(S)} = \sum_{n=1}^{N} \lambda_{agg,n} \omega_n^{(E)}, \tag{3}$$

where $\omega^{(S)}$ is the updated global model, $N$ is the number of edge servers covered by the central server, $\omega_n^{(E)}$ is the model weights of edge server $n$, and $\sum_{n=1}^{N} \lambda_{agg,n} = 1$.

### B. Problem Definition

The AHFL urgently needs to address the following critical problems, which directly impact its efficiency, scalability, and ability to achieve an optimal global model in distributed environments:

Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.

- **Optimality of the Global Model**. Due to environmental differences, data typically do not follow an IID distribution. This data heterogeneity causes local models to deviate from the global model, challenging achieving an optimal global model. The discrepancy between the local model trained on each client's data and the global model can be quantified by the following objective function:

$$f_d = \min \sum_{i=1}^{N} \sum_{j=1}^{K} \|\lambda_i(y=j) - \lambda(y=j)\|, \quad (4)$$

where $\lambda_i(y=j)$ represents the local label distribution at client $i$, and $\lambda(y=j)$ represents the global label distribution. The function $f_d$ measures the difference between local and global model distributions, which arises due to the non-IID nature of the data. Minimizing this difference is crucial to ensure the global model converges to an optimal solution.
- **Handling the Obsolescence Effect**. Asynchronous updates may delay sending local models to the edge server, causing the obsolescence effect. The impact of these delays on the convergence of the global model is critical, as outdated updates may hinder the model's accuracy. The obsolescence effect at time step $t$ for client $c_t$ can be expressed as:

$$s(t - \tau^{c_t}) = (1 + t - \tau^{c_t})^{-\alpha}, \quad (5)$$

where $t$ is the current time step, $\tau^{c_t}$ is the last synchronization time for client $c_t$, and $\alpha$ is a decay factor that models how the obsolescence effect decreases over time. The challenge is to manage and mitigate the effect of obsolescence in model aggregation, ensuring that outdated updates do not degrade the quality of the global model.
- **Efficient Aggregation of Local Models**. Efficiently aggregating models from clients with asynchronous updates is essential for minimizing the obsolescence effect and achieving fast convergence. The global model is updated based on aggregating local models from each client. The challenge lies in handling the updates from clients with different synchronization times. The aggregation of local models is given by:

$$\omega_{t+1} = \omega_t - \eta_g \sum_{i=1}^{M} \lambda_i \nabla f_i(\omega_t), \quad (6)$$

where $\omega_{t+1}$ is the updated global model, $\omega_t$ is the current global model, $\eta_g$ is the global learning rate, $\lambda_i$ is the weight for client $i$'s contribution to the global model, and $\nabla f_i(\omega_t)$ is the gradient computed by client $i$ based on its local data. The goal is to optimize the aggregation process so that the global model converges efficiently.

## IV. Design of AchiFL

The challenges confronting AHFL are the obsolescence effect induced by the asynchronous mechanism and the exacerbation of non-IID data issues resulting from the hierarchical structure. We design `AchiFL` to tackle these challenges.

### A. System Overview

In the proposed `AchiFL`, the central server initializes and distributes parameters to each client, then clusters clients based on their data. Afterwards, the edge servers prune and aggregate the gradient parameters uploaded by the clients into edge models, which are then uploaded to the central server. The central server weighted aggregates the received edge models to generate a global model, which is then distributed to each client. After that, repeat steps 2 and 3 until the global model performance converges. Specifically, as shown in Fig. 2, `AchiFL` consists of three steps:

- **Step 1: Birch clustering clients**. We use the clustering algorithm, with the DK Divergence (DKD) as the evaluation metric, to group clients with similar data characteristics under the same edge server.
- **Step 2: Edge server aggregation local models**. The edge server begins edge model aggregation. Furthermore, the edge server performs gradient pruning and growth operations on the clients that complete training and upload local models within the $T_0$ clock. Gradients in the same direction as the global model are grown, while gradients in the opposite direction are pruned.
- **Step 3: Central server aggregation edge models**. The edge servers upload the edge models to the central server for global model aggregation. The aggregation uses three parameters: the number of clients participating in training under the edge server, the accuracy of the local model trained by the edge server, and the obsolescence effect of the local model trained by the edge server. These parameters are used to calculate dynamic weights based on accuracy and for weighted averaging to aggregate the global model. The obsolescence effect is used as a penalty term so that the edge server pays more attention to clients with severe obsolescence effect.

The workflow of `AchiFL` is presented in Algorithm 1. The following introduces the detailed steps of `AchiFL`.

### B. Birch Clustering Clients

After initializing the client, we use the Balanced Iterative Reducing and Clustering using Hierarchies (BIRCH) clustering algorithm [69], with DKD as the evaluation metric, to cluster clients with similar data characteristics under the same edge server. The BIRCH clustering algorithm is highly efficient in UAV scenarios with numerous clients. It can aggregate clients with similar data distributions under the same edge server, making the pruning and growth of client gradients more conducive to edge model aggregation.

Assuming there are $k$ clients, each with its dataset $\mathbf{D}_i$. The BIRCH algorithm aims to cluster the data from these clients into several clusters. Let $\mathcal{C}_i$ represent the $i$-th cluster, $\mathbf{P}_i$ denote the probability distribution of $\mathcal{C}_i$, and $\mathbf{Q}_i$ be the probability distribution of the global cluster. The DKD is defined as:

$$D_{\mathrm{KL}}(\mathbf{P}_i \| \mathbf{Q}_i) = \sum_{x \in \mathcal{C}_i} \mathbf{P}_i(x) \log \frac{\mathbf{P}_i(x)}{\mathbf{Q}_i(x)}, \quad (7)$$

Here, $x$ represents a sample in the cluster. In the BIRCH algorithm, $\mathbf{Q}_i$ is typically chosen as the probability distribution

Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.

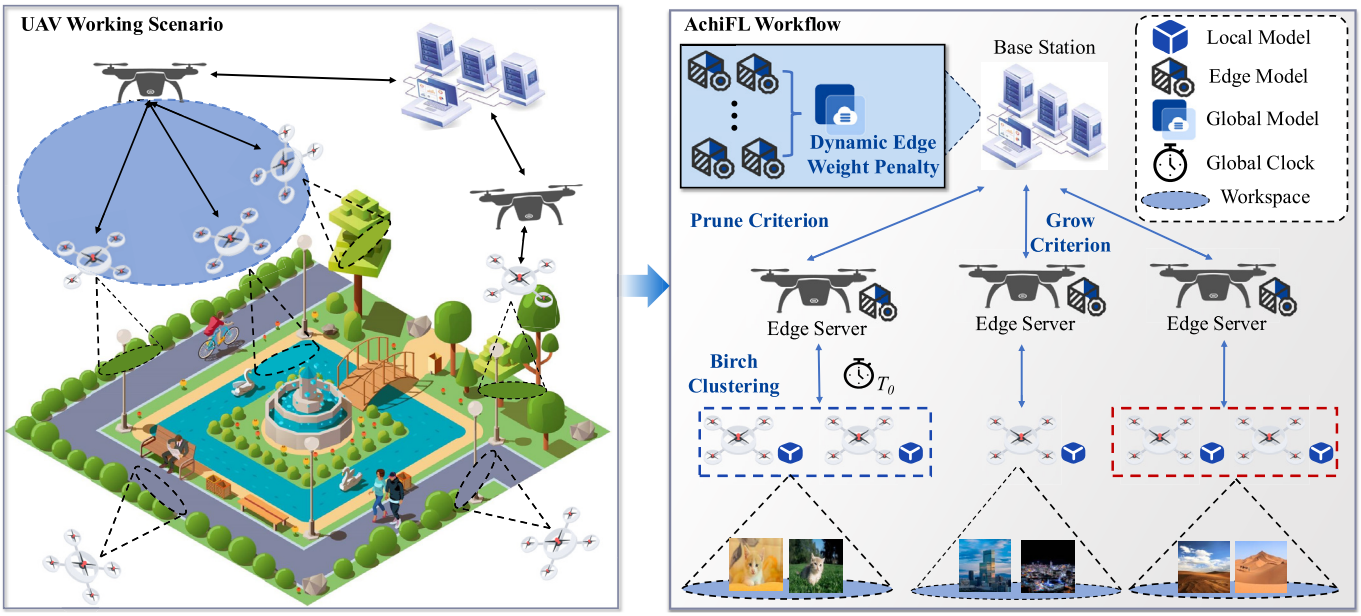



Fig. 2. Overview of our proposed AchiFL.

of the global cluster. Each client can calculate the DKD between its cluster $\mathcal{C}_i$ and the global cluster $\mathbf{Q}_i$ and then use these divergences as metrics to measure the quality and stability of the clusters.

Given the dynamic nature of drone movements, the data distribution at each client can change over time. To handle these fluctuations, we periodically update the clusters formed by BIRCH to reflect the latest data distribution. If significant changes in data characteristics are observed, a new round of clustering is triggered to ensure that clients are grouped based on their current data distribution. This approach helps mitigate the impact of rapidly changing data distributions caused by the movement of drones, ensuring more effective model training and aggregation.

### C. Edge Server Aggregation Local Models

Clients assigned to the same edge server begin training their local models independently. Once a client finishes training within the specified time frame, it can upload its model asynchronously. The edge server applies specific operations to the received models: it prunes gradients that oppose the direction of the global model while enhancing those that align with it. After these adjustments, the edge server aggregates the processed models into a unified edge model.

Only clients who have completed their training within the defined time $T_0$ limit can contribute to this aggregation. Clients that miss the cutoff time will not be included in the aggregation process.

$$\omega^{(E)} = \sum_{k=1}^{K} \begin{cases} \lambda_{agg,k}\omega_k, & \text{if } T_k \leq T_0 \\ 0, & \text{otherwise} \end{cases} \tag{8}$$

Here, the clock $T_0$ is dynamically adjusted based on the impact of the first round of client-synchronized gradient submissions and the subsequent asynchronous gradient uploads.

$$T_0 = t_0 + \rho\Delta_{\text{accuracy}}(t_i - t_j), \tag{9}$$

$t_0$ is the time of the first round of client synchronized gradient submission, $\rho$ is a factor controlling how sensitive the adjustment is to accuracy fluctuations, $\Delta_{\text{accuracy}}(t_i - t_j)$ represents the change in global model accuracy between time steps $t_i$ and $t_j$, which is used to adjust the clock dynamically.

The value of $\rho$ determines how sensitive the control clock is to the fluctuations in the accuracy of the model during training. If $\rho$ is set to a high value, the global clock adjusts more quickly in response to accuracy fluctuations, allowing faster adaptation to changes in client training progress. However, excessively high values may result in unstable clock adjustments, leading to inefficient model aggregation. On the other hand, small values of $\rho$ make the clock less sensitive to accuracy fluctuations, potentially causing slower convergence but with more stable updates. Based on our experiments, we suggest that $\rho$ should typically lie within the range of $[0, 1]$, with values closer to 0 offering more stable but slower convergence, and values closer to 1 providing faster adaptation but potentially less stability.

This adjustment also prevents the delay in edge model aggregation caused by a single client's failure or weak computational capability.

Afterwards, the edge server prunes and grows the gradients of the local models uploaded by clients that completed training within the $T_0$ clock. In the edge server aggregation local model phase, we still consider the parameters that are held by those clients who do not participate, as we find that updating the edge model only by partially selected clients' learning results may lead to the unstable performance of the edge model [70].

*1) Prune Criterion:* On the client $c$-side, we select the parameters with local pseudo-gradient of opposite sign from the global direction and then sort them by their weight absolute value in ascending order. We first prune the first $\lambda_{prune}$ of them.

$$K_{\text{guided}} = \text{AchiTopS}(-|\theta^l_{[i|d^r=-\text{sign}(\Delta^r_c)]}|, \lambda_{prune}k), \tag{10}$$

Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.

TABLE II
USED SYMBOLS IN ACHIFL

| Symbol | Description | Symbol | Description |
|---|---|---|---|
| $\omega_k$ | Local model of client $k$ | $\omega_n^{(E)}$ | Edge server model at $n$ |
| $\omega_t$ | Global model at iteration $t$ | $\omega^{(S)}$ | Global model |
| $M$ | Total number of clients | $N$ | Number of edge servers |
| $\lambda_{agg,k}$ | Weight of client $k$ in aggregation | $\lambda_{agg,n}$ | Weight of edge server $n$ in global aggregation |
| $\eta_g$ | Global learning rate | $\alpha$ | Decay factor for obsolescence |
| $T$ | Total number of training rounds | $t$ | Training round index |
| $F(\omega)$ | Global loss function | $F^*$ | Optimal global loss |
| $\sigma_l^2$ | Local gradient variance (client index) | $\sigma_g^2$ | Global gradient variance |
| $L$ | Smoothness constant for $F(\omega)$ | $s(t-\tau^{c_t})$ | Obsolescence effect at time $t$ for client $c_t$ |

where we treat the layer parameter as a one-dimensional vector, $i$ is the neuron/parameter index in the vector, and the function AchiTopS returns the indices of the top $s$-elements in the vector. $\Delta_c^r = \theta_c^{r,t} - \theta_c^{r,0}$ denotes the local pseudo-gradient of client $c$ at the current iteration step $t$ of round $r$, $\lambda_{prune}$ is a hyper-parameter that controls the proportion of pruning guided by the gradient congruity. The motivation here is that the neurons where the associated global and local gradients are incongruent (the signs of global and local gradients are different) are less likely to be generalized across heterogeneous data. For the remaining neurons neurons not in $K_{\text{guided}}$ to be pruned, we use the magnitude of the parameters only.

$$K_{\text{mag}} = \text{AchiTopS}(-|\theta^l_{[i\notin K_{\text{guided}}]}|, (1-\lambda_{prune})k), \tag{11}$$

By pruning in this order, we prioritize pruning those with opposite directions while generally pruning by weight magnitude. Finally, we pruned the parameters with indices in $K_{\text{guided}} \cup K_{\text{mag}}$.

*2) Grow Criterion:* Similar to the pruning stage, we adopt a prioritized approach to growing neurons. Initially, we grow the neurons with the most significant magnitude of loss gradient and the same learning direction sign as the global direction.

$$\hat{K}_{\text{guided}} = \text{AchiTopS}(|\nabla^l_{[i|d^r=\text{sign}(\Delta_c^{r,t})]}|, \lambda_{grow}\hat{k}), \tag{12}$$

where we grow the neurons that the associated direction of global and local gradient are congruent $(i|d^r = \text{sign}(\Delta_c^r))$, where $\Delta_c^r = \theta_c^{r,t} - \theta_c^{r,0}$ is the pseudo gradient of client $c$ at current iteration $t$ of round $r$, and $i$ is the neuron index. Such neurons are safer to grow as we expect them to be better generalized across clients with heterogeneous data. For the remaining neurons (neurons not in $K_{\text{guided}}$) to be grown, we use the loss gradient magnitude given as:

$$\hat{K}_{\text{mag}} = \text{AchiTopS}(|\nabla^l_{[i\notin \hat{K}_{\text{guided}}]}|, (1-\lambda_{grow})\hat{k}), \tag{13}$$

In the growing step, we grow neurons with indices in $\hat{K}_{\text{guided}} \cup \hat{K}_{\text{mag}}$.

### D. Central Server Aggregation Edge Models

Edge servers upload edge models to the central server for global model aggregation. The central server calculates dynamic weights based on accuracy using three parameters: the number of clients participating in the training under the edge server, the accuracy of the local model trained by the edge server, and the local model's obsolescence effect. These

**Algorithm 1** The Training Process of AchiFL

**Input:** Number of data $\mathbf{X}_k$, set of clients $M$ and set of edge servers $N$;
**Output:** Global model $\omega_t^{(S)}$;
The central server propagates its initial model to $M$ clients;
**BIRCH Clustering Clients;**
$D_{\text{KL}}(\mathbf{P}_i \| \mathbf{Q}_i) \longleftarrow \sum_{x\in\mathcal{C}_i} \mathbf{P}_i(x) \log \frac{\mathbf{P}_i(x)}{\mathbf{Q}_i(x)}$;
**Update Local Model (K);**
**for** *local training epoch* $h = 1, 2, \ldots, H$ **do**
  **for** *each local batch* $b \in \boldsymbol{\beta}$ **do**
    $\omega_k \longleftarrow \omega_k - \eta_g \frac{1}{\beta} \sum_{x_k\in\mathbf{X}_k} \Delta f(\omega_k; x_k)$;
  **end**
**end**
**Edge Server Aggregation Local Model;**
Calculate clock $T_0$;
**for** *each edge server* $n$ **do**
  **for** *each client* **do**
    $\omega^{(E)} \longleftarrow \sum_{k=1}^{K} \begin{cases} \lambda_{agg,k}\omega_k, & \text{if } T_k \le T_0 \\ 0, & \text{otherwise} \end{cases}$;
    **if** *The gradient of the local model uploaded by the client is the opposite of the global model* **then**
      **Prune criterion;**
      $AchiTopS(-|\theta^l_{[i|d^r=-\text{sign}(\Delta_c^r)]}|, \lambda_{prune}k)$;
      $AchiTopS(-|\theta^l_{[i\notin K_{\text{guided}}]}|, (1-\lambda_{prune})k)$;
    **end**
    **else**
      **Grow criterion;**
      $AchiTopS(|\nabla^l_{[i|d^r=\text{sign}(\Delta_c^{r,t})]}|, \lambda_{grow}\hat{k})$;
      $AchiTopS(|\nabla^l_{[i\notin \hat{K}_{\text{guided}}]}|, (1-\lambda_{grow})\hat{k})$;
    **end**
  **end**
  $\omega^{(E)}_{\text{new},t} \longleftarrow \sum_{n=1}^{N} \frac{c_n}{C} \omega^n_{\text{new},t}\varphi$;
**end**
**Central Server Aggregation Edge Model;**
$\omega_t^{(S)} \longleftarrow (1-\beta_t)\omega_{t-1}^{(S)} + \beta_t \omega^{(E)}_{\text{new},t}$;

Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.

weights are then used for weighted averaging to aggregate the global model.

The central server waits for the edge servers to report their updates in each epoch. Each edge server's model is weighted based on three factors: the number of clients it has, the accuracy of the edge model, and the clients' training speed.

$$\omega_{\text{new},t}^{(E)} = \sum_{n=1}^{N} \frac{c_n}{C} \omega_{\text{new},t}^{n} \varphi, \tag{14}$$

$$\text{s.t. } \varphi = \left\{ \mathbf{1}\left(t < T_0\right) \times e\left(\text{acctrain}_t^n\right) + \mathbf{1}\left(t \geq T_0\right) \right\}. \tag{15}$$

The term $c_n \setminus C$ normalizes each edge server by its number of clients. $\mathbf{1}(\cdot)$ is the identity function, which is 1 when the argument is True, else 0. This rewards edge servers, which return results within a latency threshold of $T_0$. The function $e(\text{acctrain}_t^n) = 1 - \text{acctrain}_t^n$ considers this and thus gives more importance to the edge servers with a low training accuracy with the current state of the edge model. Intuitively, when an edge server's training accuracy is high, the global model has already learned the features corresponding to that edge server's data, and our global model can focus on other edge servers to learn their features.

Let us consider how the global model aggregation handles stragglers and penalizes outdated updates from edge servers. The weighting factor is determined in this process.

$$\beta_t = \beta \times s(t - \tau^{c_t}), \tag{16}$$

We calculate $\omega_t^{(S)}$, the global model at epoch $t$.

$$\omega_t^{(S)} = (1 - \beta_t)\omega_{t-1}^{(S)} + \beta_t \omega_{\text{new},t}^{(E)}, \tag{17}$$

We moderate the mixing hyperparameter $\beta \in (0, 1)$. Here $t - \tau^{c_t}$ captures how delayed the burst is, and we calculate the obsolescence effect of the burst as $s(t - \tau^{c_t}) = (1 + t - \tau^{c_t})^{-\alpha}$, where $\tau^{c_t} = \text{avg}\{\tau^i, \forall i \in (1, 2, \ldots, i, \ldots, K)\}$, which adaptively changes the mixing parameter $\beta_t^{c_t}$. This function's general form is monotonically an obsolescence effect.

The weight penalty mechanism adjusts the influence of edge servers on the global model by accounting for the delay caused by both active and passive factors. Active factors, such as limited computational resources on the edge servers, can lead to delays in processing and model aggregation. In this case, the delay is directly related to the computational capability of the edge server and can be mitigated by optimizing the server's resources or increasing computational power. On the other hand, passive factors like poor communication channel conditions can also cause delays, as they hinder the timely transmission of model updates between the edge servers and the central server. These delays can be addressed by improving network quality or using communication-efficient protocols. By distinguishing between these two types of delay, the weight penalty mechanism can adaptively adjust the contribution of each edge server based on its specific delay characteristics, ensuring that the global model remains up-to-date and converges efficiently.

## V. Convergence Analysis

Given the assumptions of unbiased stochastic gradients, bounded gradient variance, and the smoothness of the global loss function, the convergence of `AchiFL` is guaranteed. The `AchiFL` converges by analyzing the impact of obsolescence effect and non-IID data. The obsolescence effect is bounded by a decay factor, which reduces the influence of outdated updates on the global model. Meanwhile, the non-IID data increase gradient variance, slowing down convergence. The convergence rate is proven through iterative bounding of the global gradient norm and applying properties of smooth convex optimization. For sufficiently small global learning rates, the global gradient norm decreases over time, with a final convergence bound that considers the effects of gradient variance, learning rates, and the obsolescence effect. Due to the space limitations, we present the detailed convergence proofs in Section I of our submitted Supplementary Material.

## VI. Performance Evaluation

This section outlines the experimental setup, including the datasets, evaluation metrics, and simulation conditions used to assess the performance of `AchiFL`. Following this, we present the evaluation results, highlighting the key findings from the experiments to compare `AchiFL` against state-of-the-art methods regarding accuracy, communication overhead, time expenditure, and convergence behaviour.

### A. Experimental Setup

*1) Datasets:* We evaluate `AchiFL` using two popular datasets, MNIST and Fashion-MNIST, as described below.

- **MNIST**: The MNIST dataset is a benchmark for handwritten digit classification. It contains 70,000 grayscale images of size $28 \times 28$ pixels. It includes 10 classes representing digits 0-9.
- **Fashion-MNIST**: The Fashion-MNIST dataset is a more challenging alternative to MNIST. It contains 70,000 $28 \times 28$ grayscale images of clothing items across 10 categories, such as T-shirts, dresses, and sneakers.

The MNIST and Fashion-MNIST datasets were chosen because they are relevant to simulating the challenges UAV face in resource-constrained environments. UAV often operate in remote or dynamic settings where computational power, storage, and communication bandwidth are limited due to lightweight design, battery constraints, and fluctuating network conditions. These limitations necessitate efficient data processing and minimized communication overhead. Both MNIST and Fashion-MNIST feature low-resolution grayscale images, which are computationally lightweight and reduce communication costs between UAV and servers. Fashion-MNIST, with its diverse clothing categories, further represents the data heterogeneity that UAV may encounter in real-world missions, where environmental and task-specific variations lead to non-IID data distributions.

*2) Baselines:* We compare `AchiFL` with FedAvg, AFed, HFed, DHFed, and AHFed methods. We adopt their open-source code, adhering strictly to the original parameter settings specified in their paper.

- **FedAvg** [12]: A fundamental baseline model aggregates the weights of selected clients in an averaged manner.

Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON BETWEEN ACHIFL AND THE STATE-OF-THE-ART

| Dataset | Method | Level of non-IID data | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | $\alpha$ = 1.0 | | | $\alpha$ = 0.4 | | | $\alpha$ = 0.2 | | |
| | | Acc | Time | Comm | Acc | Time | Comm | Acc | Time | Comm |
| **MNIST** | FedAvg [12] | 89.75 | 81.82 | 8.00 | 71.98 | 77.40 | 8.00 | 55.51 | 78.59 | 8.00 |
| | AFed [71] | 89.33 | **70.01** | 7.14 | 71.65 | **70.01** | 7.14 | 55.62 | **70.01** | 7.14 |
| | HFed [22] | 87.71 | 96.13 | 1.52 | 74.69 | 92.16 | 1.52 | 44.19 | 93.73 | 1.52 |
| | DHFed [63] | 88.07 | 70.02 | 1.25 | 77.71 | 70.02 | 1.25 | 45.28 | 70.02 | 1.25 |
| | AHFed [64] | 91.23 | 70.02 | 1.25 | 80.19 | 70.02 | 1.25 | 53.24 | 70.02 | 1.25 |
| | AchiFL | **92.47** | 80.32 | **1.22** | **83.29** | 80.23 | **1.22** | **67.42** | 80.15 | **1.22** |
| **FASHION** | FedAvg [12] | 68.74 | 77.76 | 8.00 | 47.2 | 76.71 | 8.00 | 33.59 | 76.94 | 8.00 |
| | AFed [71] | 68.79 | **70.01** | 7.14 | 47.61 | **70.01** | 7.14 | 32.74 | **70.01** | 7.14 |
| | HFed [22] | 65.58 | 121.76 | 1.52 | 50.10 | 120.99 | 1.52 | 31.48 | 121.27 | 1.52 |
| | DHFed [63] | 68.02 | 70.02 | 1.25 | 40.40 | 70.02 | 1.25 | 27.58 | 70.02 | 1.25 |
| | AHFed [64] | 69.77 | 70.02 | 1.25 | 52.12 | 70.02 | 1.25 | 39.37 | 70.02 | 1.25 |
| | AchiFL | **71.94** | 80.32 | **1.22** | **57.13** | 80.41 | **1.22** | **44.71** | 80.27 | **1.22** |

* $\alpha$ represents the degree of non-IID data. When $\alpha$=1.0, it indicates that the data is similar and uniformly distributed. Values of $\alpha$ less than 1.0 indicate non-IID characteristics, with $\alpha$ denoting the percentage of total labels represented by the number of category labels held by a client. The lower the $\alpha$, the higher the degree of non-IID data. Acc represents the accuracy of the final training round, measured in percentage (%). Time is the average time required to complete one global model iteration during training, measured in seconds (S). Comm represents the average communication overhead during one global model iteration, measured in (MB).

- **AFed** [71]: This framework updates the central model asynchronously to address the challenges posed by varying computational loads of heterogeneous edge clients and their delays or losses.
- **HFed** [22]: It proposes a client-edge-cloud system that allows multiple edge servers to perform partial model aggregation, which enables faster model training and achieves better communication-computation trade-offs.
- **DHFed** [63]: It introduces a decentralized hierarchical network architecture that leverages a two-stage model aggregation paradigm, with edge servers employing asynchronous model averaging.
- **AHFed** [64]: It proposes a novel AHFL method that divides clients into clusters, with aggregation only occurring within each cluster.

*3) Non-IID:* In this paper, $\alpha$ quantifies the degree of non-IID data across the clients. Specifically, $\alpha$ represents the imbalance or heterogeneity in the data distributions between the clients. When $\alpha = 1.0$, it means the data across all clients are IID (identically and independently distributed), indicating no data heterogeneity. As $\alpha$ decreases, the data distributions become increasingly non-IID, meaning that clients hold data from more diverse and potentially unbalanced distributions. This data heterogeneity introduces challenges in FL, as it can cause slow convergence and lower model accuracy due to the discrepancy between local client models and the global model.

For example, when $\alpha = 0.5$, clients may hold data from only a subset of classes or with skewed class distributions, while $\alpha = 0.2$ indicates a more severe data heterogeneity where each client may only hold a few classes of data, leading to greater variability in the model updates. These different levels of non-IID data cause greater difficulty in achieving global model convergence, as clients' local models may diverge more significantly from the global model.

*4) Simulation:* To simulate a real UAV scenario, we make the following setup.

- **Scenario Simulation**. Six GeForce RTX 3060 6G devices were utilized to simulate a UAV scenario: one acting as the central server, four as edge servers, and one simulating multiple UAV clients. Each client was allocated a different amount of data, with volumes varying randomly within a 100-fold range to replicate data heterogeneity across UAV operating in various environments. Computational resources were varied by adjusting each device's CPU and GPU capacities to reflect the UAV' differing hardware capabilities. Communication delays were modelled by assigning random latencies and bandwidth constraints to each client's connection with the edge and central servers, simulating realistic network conditions. Each client's computation was represented by multiple threads to simulate parallel tasks such as data processing, model updating, and communication handling, reflecting the asynchronous nature of UAV operations.
- **Asynchronous Simulation**. To determine the global clock $T_0$, we conduct pre-experiments to ensure that we wait for clients with similar training times and exclude excessively outdated clients. This approach helps us find a balance where the global clock is flexible and lenient. After completing training, the device simulating the clients transmits the trained local models via network communication to the device simulating the edge servers. Similarly, the edge servers transmit the aggregated edge models to the central server.

*5) Client Selection Strategy:* In the experiments, a dynamic client selection strategy is used to optimize the training process. At the beginning of each training round, a subset of clients is randomly selected to participate in model updates. The client selection is based on the following criteria.

Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.

- **Computational Capacity**. Clients with higher computational power are preferred to ensure faster model updates and reduce training time.
- **Data Similarity**. Clients whose data distribution is more similar to the global model are prioritized to minimize the impact of non-IID data. A clustering algorithm is used to group clients with similar data characteristics, and each group contributes updates to the global model.
- **Network Conditions**. Clients with better network conditions (e.g., lower latency) are chosen to ensure timely data transmission for faster model aggregation.

The selection strategy is updated dynamically throughout the training process, with clients being replaced or re-selected based on their performance in previous rounds, ensuring that the global model benefits from a diverse and representative set of client updates.

*6) Hyperparameters and Evaluation:* We utilize 80 clients to simulate devices participating in real UAV scenarios, with 60% of the clients selected for training in each epoch. Clients use a clustering algorithm to group clients with similar data distributions under the same edge server. Each client completes five epochs of local training per round. We set the learning rate to 0.1 and the local mini-batch size to 10. We set up four edge servers and one central server. After training, the overall performance is measured by the accuracy of the global model aggregated by the central server on the test dataset. The size of the transmitted model is used as the communication overhead, and the completion time of model training is recorded. The accuracy is the model's accuracy upon convergence, while the communication overhead and completion time are the averages over the training iterations. To ensure a fair comparison of different methods, we meticulously designed several techniques, including seeding, saving, and restoring random numbers to enhance the reproducibility of experiments. Additionally, we conducted multiple experiments and averaged the results to ensure stability and reliability.

### B. Overall Performance

*1) Higher Accuracy:* Table III presents a comparison between AchiFL and other state-of-the-art methods on the MNIST and Fashion-MNIST datasets, with the advantages of AchiFL highlighted in bold. The table shows the accuracy of each method under varying levels of non-IID data. In uniform data scenarios (where $\alpha = 1.0$), AchiFL consistently outperforms other methods. This can be attributed to its clustering pruning mechanism, which adjusts local models of clients with gradients opposite to the global model, leading to improved accuracy. Furthermore, when introducing non-IID data, AchiFL significantly outperforms existing methods, demonstrating its robustness in handling data heterogeneity.

In the MNIST dataset, when $\alpha = 1.0$ (uniform data distribution), AchiFL improves accuracy by 1.24% compared to AHFed. In non-IID scenarios, AchiFL shows even more significant improvements of 3.10% and 14.18% for different values of $\alpha$. Similarly, for the Fashion-MNIST dataset, AchiFL improves accuracy by 2.17% compared to AHFed in uniform data scenarios, with improvements of 5.01% and 5.34% in non-IID data scenarios. Unlike AHFed, which only utilizes a clustering strategy to address non-IID data, AchiFL employs a more effective pruning and growth strategy that fundamentally addresses the non-IID data issues, resulting in substantial improvements in accuracy across all datasets.

TABLE IV
COMMUNICATION OVERHEAD OF DIFFERENT MODULES

| Dataset | Method | Communications Overhead (MB) | | |
|---|---|---|---|---|
| | | Client | Edge | Central |
| **MNIST** | FedAvg | 4.00 | - | 4.00 |
| | AFed | 3.57 | - | 3.57 |
| | HFed | 0.51 | 0.51 | 0.51 |
| | DHFed | 0.42 | 0.42 | 0.42 |
| | AHFed | 0.42 | 0.42 | 0.42 |
| | AchiFL | **0.41** | **0.41** | **0.41** |
| **FASHION** | FedAvg | 4.00 | - | 4.00 |
| | AFed | 3.57 | - | 3.57 |
| | HFed | 0.51 | 0.51 | 0.51 |
| | DHFed | 0.42 | 0.42 | 0.42 |
| | AHFed | 0.42 | 0.42 | 0.42 |
| | AchiFL | **0.41** | **0.41** | **0.41** |

* - indicates that the module does not have the value.

*2) Lower Communication Overhead:* Table IV presents the communication overhead for various modules across the MNIST and Fashion-MNIST datasets. Communication overhead refers to the size of model parameters transmitted during training. The table clearly shows that the communication overhead is reduced in methods with hierarchical structures or asynchronous mechanisms. AchiFL utilizes both clustering and pruning strategies to significantly reduce the amount of data transmitted compared to state-of-the-art methods such as FedAvg, showcasing its efficiency in minimizing communication costs. This reduction is achieved by aggregating clients with similar data distributions, which helps reduce the overall communication required for training.

In the MNIST dataset, FedAvg and AFed have a communication overhead of 4.00 MB, while methods with hierarchical structures like HFed, DHFed, and AHFed reduce this overhead to 0.51 MB, 0.42 MB, and 0.42 MB respectively. AchiFL further reduces this overhead to 0.41 MB on all modules (client, edge, and central), demonstrating its effectiveness in reducing communication costs. Similarly, in the Fashion-MNIST dataset, AchiFL shows the same communication overhead of 0.41 MB across all modules. At the same time, state-of-the-art methods like FedAvg and AFed have significantly higher overheads of 4.00 MB and 3.57 MB. This demonstrates the advantage of AchiFL in minimizing communication costs while efficiently handling client data.

*3) More Time Expenditure:* Table V presents the time expenditure of each module across different methods on the MNIST and Fashion-MNIST datasets. The time consumed in asynchronous learning is primarily determined by the client's training time, and it can vary due to different methods. The time required for the client module can significantly influence the overall time, especially when specific clients' training times are much longer than others. AFL methods, such as

Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.

TABLE V
EXPENDITURE OF TIME FOR DIFFERENT MODULES

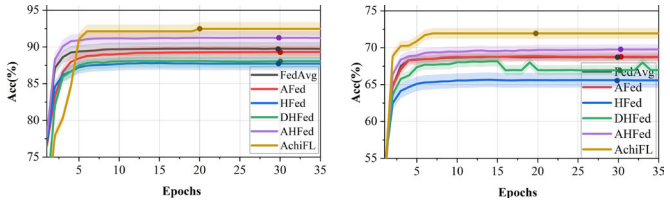
| Dataset | Method | Expenditure of Time (S) | | |
|---|---|---|---|---|
| | | Client | Edge | Central |
| **MNIST** | FedAvg | $80.20 \pm 1.62$ | - | 0.01 |
| | AFed | 70.00 | - | 0.01 |
| | HFed | $94.13 \pm 1.99$ | 0.01 | 0.01 |
| | DHFed | 70.00 | 0.01 | 0.01 |
| | AHFed | 70.00 | **0.01** | **0.01** |
| | AchiFL | **70.00** | $5.13 \pm 1.72$ | $8.40 \pm 1.80$ |
| **FASHION** | FedAvg | $77.23 \pm 0.53$ | - | 0.01 |
| | AFed | 70.00 | - | 0.01 |
| | HFed | $121.36 \pm 0.39$ | 0.01 | 0.01 |
| | DHFed | 70.00 | 0.01 | 0.01 |
| | AHFed | 70.00 | **0.01** | **0.01** |
| | AchiFL | **70.00** | $5.07 \pm 0.07$ | $6.11 \pm 0.09$ |

[1] - indicates that the module does not have the value, ± indicates the range of values.
[2] The reason for not using the ± notation is that they involve asynchronous clock mechanisms.

AHFL and AHFed, benefit from the dynamic adjustment of the clock $T_0$, which allows better synchronization and reduces the time spent compared to other approaches. Notably, the server-side computation time is generally minimal, contributing less to the overall expenditure.

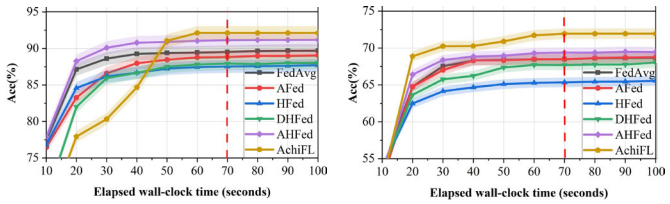
Regarding specific results, the time expenditure for AchiFL is higher than other methods due to the clustering pruning and weight penalty mechanisms. For the MNIST dataset, AchiFL spends 70.00 seconds on the client side, $5.13 \pm 1.72$ seconds on the edge side, and $8.40 \pm 1.80$ seconds on the central server, leading to a higher total time consumption compared to FedAvg and other methods. Similarly, for the Fashion-MNIST dataset, AchiFL shows an expenditure of 70.00 seconds for the client side, $5.07 \pm 0.07$ seconds for the edge side, and $6.11 \pm 0.09$ seconds for the central server. The additional time spent on the edge and central server modules in AchiFL is due to the added mechanisms that enhance its accuracy and performance, albeit at the cost of increased processing time.

AchiFL demonstrates improved performance in terms of model accuracy and convergence under non-IID data, but it comes at the cost of increased computation time at the edge server. This is primarily due to the additional mechanisms incorporated in AchiFL. The edge server in AchiFL not only performs standard model aggregation but also executes gradient pruning and growth operations, which aim to reduce the impact of contradictory gradients and enhance those aligned with the global model. These operations are crucial for accelerating convergence under non-IID conditions but naturally increase the processing time compared to simpler aggregation schemes used in baseline methods. Furthermore, AchiFL handles non-IID data by grouping clients with similar data distributions, which leads to more heterogeneous client models. This heterogeneity requires more time to aggregate effectively, unlike baseline methods that assume IID data, where model aggregation is simpler and faster. Additionally, AchiFL's dynamic adjustment of the global clock to accommodate fluctuations in model accuracy introduces further complexity and




(a) MNIST (b) FASHION-MNIST

Fig. 3. Figures (a) and (b) show the accuracy variations of different methods at $\alpha = 1.0$. The lines represent the accuracy of the global model over training epochs for each method. The shaded area represents the standard deviation (or confidence interval) of the accuracy across multiple runs, illustrating the variability and stability of each method during training.




(a) MNIST (b) FASHION-MNIST

Fig. 4. Figures (a) and (b) show the accuracy variations of different methods over time as the clock advances, with $\alpha$ set to 1.0. The red dashed line indicates that the accuracy of each method stabilizes when $T_0 = 70$. The lines represent the accuracy of the global model over training epochs for each method. The shaded area represents the standard deviation of the accuracy across multiple runs, illustrating the variability and stability of each method during training.

computational overhead. While this dynamic clock adjustment enhances synchronization and training efficiency, especially in more complex scenarios, it also contributes to the increased processing time at the edge server.

### C. Sensitivity Evaluation

*1) Faster Convergence:* Fig. 3a and Fig. 3b show the accuracy variations of different methods across different datasets under uniform data distribution. The lines represent the accuracy of the global model over training epochs for each method. The shaded area represents the standard deviation (or confidence interval) of the accuracy across multiple runs, illustrating the variability and stability of each method during training. Due to the weight penalty mechanism considering the impact of the obsolescence effect on model aggregation and convergence, AchiFL's accuracy stabilizes after about 20 epochs. In comparison, other methods do not show significant changes until around 30 epochs. When $\alpha = 1.0$, indicating a uniform data distribution, the influence of the non-IID data issues on convergence is avoided. Although the changes in accuracy are not visually apparent in the figure, the raw data indicates that the weight penalty mechanism accelerates AchiFL's convergence, thereby addressing the obsolescence effect.

*2) Reasonable Clock:* Fig. 4a and Fig. 4b illustrate how the accuracy of different methods evolves on the MNIST and Fashion-MNIST datasets as the global clock time progresses. These plots highlight the importance of setting an appropriate global clock to balance the synchronization of client updates. The lines represent the accuracy of the global model over training epochs for each method. The shaded

Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
ACHIFL FOR DIFFERENT NUMBER OF CLIENTS ($\alpha = 1.0$) ON MNIST AND FASHION-MNIST DATASETS

| Metrics | MNIST Dataset | | | |
|---|---|---|---|---|
| | 80 | 200 | 300 | 400 |
| Acc (%) | 92.47 | 93.14 | 94.03 | 94.76 |
| Time (S) | 80.32 | 83.94 | 89.13 | 94.41 |
| Comm (MB) | 1.22 | 1.67 | 1.85 | 2.13 |
| Metrics | FASHION-MNIST Dataset | | | |
| | 80 | 200 | 300 | 400 |
| Acc (%) | 71.94 | 72.54 | 73.54 | 74.34 |
| Time (S) | 80.27 | 83.91 | 89.25 | 94.63 |
| Comm (MB) | 1.22 | 1.67 | 1.85 | 2.13 |

area represents the standard deviation of the accuracy across multiple runs, illustrating the variability and stability of each method during training. Specifically, a well-calibrated global clock ensures that clients with minor update discrepancies are adequately synchronized while preventing significantly outdated models from unduly affecting the system's performance. In our experiments, when the global clock time $T_0$ reaches around 70 seconds, the accuracy of all methods converges and stabilizes, indicating that the global clock has sufficiently accommodated the variations in client update times. As a result, we set the global clock to 70 seconds to optimize the system's efficiency and accuracy.

*3) Stable Performance:* Table VI presents the performance of AchiFL with $\alpha = 1.0$ for varying numbers of clients on the MNIST dataset. The data trends reveal that the accuracy improves progressively as the number of clients increases. However, this increase in accuracy comes at the cost of additional time expenditure and communication overhead. Despite these increases, AchiFL maintains stable performance, demonstrating its robustness in handling more clients.

As the number of clients rises from 80 to 400, the accuracy improves from 92.47% to 94.76%. The time per round increases from 80.32 seconds for 80 clients to 94.41 seconds for 400 clients, and the communication overhead grows from 1.22 MB to 2.13 MB. This indicates that while AchiFL handles higher client numbers efficiently, the model's performance scales effectively without significant degradation in efficiency. These results suggest that AchiFL is a scalable and efficient framework suitable for many client sizes.

Similarly, the performance of AchiFL on the FASHION-MNIST dataset also demonstrates stable behavior across different client counts. As the number of clients increases from 80 to 400, the accuracy gradually improves from 71.94% to 74.34%. Although there is an increase in both time per round and communication overhead, the model's performance remains robust. The time per round rises from 80.27 seconds for 80 clients to 94.63 seconds for 400 clients, while the communication overhead grows from 1.22 MB to 2.13 MB. This trend mirrors the results observed in the MNIST dataset, suggesting that AchiFL performs consistently well in a variety of client scenarios, maintaining its efficiency even with a higher number of clients.

## VII. CASE STUDY

In the context of LAE networks, the deployment of UAV for applications such as precision agriculture presents unique challenges [5], [6], [7]. UAV collect a vast array of heterogeneous data due to varying environmental conditions, making it difficult to achieve model convergence and generalization across a fleet of devices. Furthermore, UAV in LAE networks typically operate in bandwidth-limited environments, where the communication overhead required for frequent model updates and data transmission can severely hinder the system's efficiency. Additionally, UAV are constrained by limited energy, necessitating optimization of both computational and communication resources to extend operational time.

The AchiFL framework is specifically designed to address these challenges through its innovative clustering-based gradient pruning and dynamic edge weight penalty mechanisms. By clustering UAV with similar data distributions at edge servers, AchiFL minimizes data discrepancies, accelerating convergence and improving model generalization. The framework also reduces communication overhead by employing hierarchical aggregation, where model updates are first aggregated at edge servers before being sent to a central server, thus alleviating bandwidth limitations. Furthermore, the dynamic edge weight penalty mechanism ensures that UAV with limited resources are not overburdened with excessive computation and communication tasks, optimizing energy usage and enhancing the system's overall efficiency.

Beyond precision agriculture, AchiFL is highly applicable in other low-altitude economy scenarios, such as smart city networks [1], [2] and disaster response [3], [4]. In smart cities, UAV are often used for tasks such as traffic monitoring, environmental sensing, and surveillance. In these contexts, AchiFL can address the same issues of data heterogeneity, communication overhead, and energy constraints, ensuring efficient and accurate model training across a distributed network of UAV. Similarly, in disaster response scenarios, where UAV are deployed to assess damage or deliver aid, the ability to handle dynamic environments and non-IID data is crucial. AchiFL's robustness in such unpredictable conditions makes it an ideal candidate for ensuring the timely and efficient processing of data in these scenarios. Furthermore, AchiFL's energy-efficient design ensures that UAV can operate for extended periods, even in energy-constrained environments, which is particularly important in mission-critical applications.

In conclusion, the AchiFL framework offers a comprehensive solution to the unique challenges faced by LAE networks, particularly in the context of UAV deployments. This framework not only improves the performance of applications in precision agriculture but also demonstrates significant potential for deployment in a wide range of other sectors, including smart cities and disaster management, where similar challenges are present.

## VIII. CONCLUSION

In this paper, we propose a novel AHFL framework, termed AchiFL, which effectively addresses key challenges in FL systems. These challenges include prolonged server waiting

Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.

times, excessive communication overhead, and issues related to non-IID data distributions and model obsolescence. Our proposed AchiFL framework introduces innovative techniques, such as a clustering-based gradient pruning mechanism and a dynamic weight penalty mechanism, to overcome these challenges. The clustering-based gradient pruning mechanism helps mitigate the negative effects of non-IID data by grouping clients with similar data distributions under the same edge server. This enables more efficient model aggregation and faster convergence. Additionally, the dynamic weight penalty mechanism accounts for the obsolescence effect during model aggregation from edge servers to the central server, ensuring that clients with outdated models do not disproportionately influence the global model. Extensive experiments conducted on the MNIST and Fashion-MNIST datasets demonstrate that AchiFL outperforms existing methods in terms of both accuracy and communication efficiency, with only a marginal increase in processing time. The experimental results confirm that AchiFL effectively addresses the issues of non-IID data and model obsolescence, providing a robust solution for Federated Learning in practical applications.

In addition to its theoretical contributions, AchiFL has significant implications for LAE scenarios, where UAV are increasingly deployed for applications such as precision agriculture, smart city infrastructure, and disaster management. The ability to address key challenges such as data heterogeneity, communication inefficiency, and energy constraints makes AchiFL particularly valuable in such resource-constrained environments. In these applications, where real-time decision-making is critical, AchiFL ensures efficient model training and aggregation while preserving data privacy and reducing communication costs. In the future, we will further explore the optimization and improvement of AchiFL in the following directions. First, adapting the framework to handle more complex multi-agent systems, where UAV collaborate with other mobile devices or sensors in dynamic environments, could improve its scalability and adaptability. Second, integrating FL with techniques like meta-learning and transfer learning could further enhance model performance, particularly in environments where labeled data is scarce or highly heterogeneous. Lastly, AchiFL could be combined with emerging technologies such as 5G networks and edge computing to address real-time communication challenges and ensure efficient processing for mission-critical applications, such as autonomous vehicles, environmental monitoring, and smart city infrastructure.

## REFERENCES


[1] E. V. Butilă and R. G. Boboc, "Urban traffic monitoring and analysis using unmanned aerial vehicles (UAVs): A systematic literature review," *Remote Sens.*, vol. 14, no. 3, p. 620, Jan. 2022.

[2] M. Balać, A. R. Vetrella, and K. W. Axhausen, "Towards the integration of aerial transportation in urban settings," *Arbeitsberichte Verkehrs-und Raumplanung*, vol. 1266, Jul. 2017.

[3] A. Rejeb, K. Rejeb, S. J. Simske, and H. Treiblmaier, "Drones for supply chain management and logistics: A review and research agenda," *Int. J. Logistics Res. Appl.*, vol. 26, no. 6, pp. 708–731, Jun. 2023.

[4] K. Kuru, D. Ansell, W. Khan, and H. Yetgin, "Analysis and optimization of unmanned aerial vehicle swarms in logistics: An intelligent delivery platform," *IEEE Access*, vol. 7, pp. 15804–15831, 2019.

[5] D. C. Tsouros, S. Bibi, and P. G. Sarigiannidis, "A review on UAV-based applications for precision agriculture," *Information*, vol. 10, no. 11, p. 349, Nov. 2019.

[6] J. Gago et al., "UAVs challenge to assess water stress for sustainable agriculture," *Agricult. Water Manage.*, vol. 153, pp. 9–19, May 2015.

[7] J. Kim, S. Kim, C. Ju, and H. I. Son, "Unmanned aerial vehicles in agriculture: A review of perspective of platform, control, and applications," *IEEE Access*, vol. 7, pp. 105100–105115, 2019.

[8] T. Templin and D. Popielarczyk, "The use of low-cost unmanned aerial vehicles in the process of building models for cultural tourism, 3D web and augmented/mixed reality applications," *Sensors*, vol. 20, no. 19, p. 5457, Sep. 2020.

[9] P. Sestras et al., "Feasibility assessments using unmanned aerial vehicle technology in heritage buildings: Rehabilitation-restoration, spatial analysis and tourism potential analysis," *Sensors*, vol. 20, no. 7, p. 2054, Apr. 2020.

[10] A. Telikani, A. Sarkar, B. Du, and J. Shen, "Machine learning for UAV-aided ITS: A review with comparative study," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 11, pp. 15388–15406, Nov. 2024.

[11] J. Shi, B. Ge, Y. Liu, Y. Yan, and S. Li, "Data privacy security guaranteed network intrusion detection system based on federated learning," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, May 2021, pp. 1–6.

[12] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artif. Intell. Statist.*, 2016, pp. 1273–1282.

[13] Y. Deng et al., "FAIR: Quality-aware federated learning with precise user incentive and model aggregation," in *Proc. IEEE Conf. Comput. Commun.*, May 2021, pp. 1–10.

[14] B. Buyukates and S. Ulukus, "Timely communication in federated learning," in *Proc. IEEE INFOCOM Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, May 2021, pp. 1–6.

[15] H. Wang, Z. Kaplan, D. Niu, and B. Li, "Optimizing federated learning on non-IID data with reinforcement learning," in *Proc. IEEE INFOCOM - IEEE Conf. Comput. Commun.*, Jul. 2020, pp. 1698–1707.

[16] Y. Zhao and X. Gong, "Quality-aware distributed computation and user selection for cost-effective federated learning," in *Proc. IEEE INFOCOM - IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, May 2021, pp. 1–6.

[17] S. Sai, S. Mishra, and V. Chamola, "Resource allocation in unmanned aerial vehicle networks: A review," *Veh. Commun.*, vol. 52, Apr. 2025, Art. no. 100889.

[18] H. Yang et al., "Dispersed computing for tactical edge in future wars: Vision, architecture, and challenges," *Wireless Commun. Mobile Comput.*, vol. 2021, no. 1, Jan. 2021, Art. no. 8899186.

[19] Y. Chen, H. Du, L. Kong, Y. Li, and R. Yang, "HyperPrism: An adaptive non-linear aggregation framework for distributed machine learning over non-IID data and time-varying communication links," in *Proc. Adv. Neural Inf. Process. Syst. 37*, 2024, pp. 11814–11836.

[20] S. Chen and B. Li, "Towards optimal multi-modal federated learning on non-IID data with hierarchical gradient blending," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2022, pp. 1469–1478.

[21] K. Hsieh, A. Phanishayee, O. Mutlu, and P. B. Gibbons, "The non-iid data distribution in federated learning: A survey and analysis," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 5293–5302.

[22] L. Liu, J. Zhang, S. Song, and K. B. Letaief, "Client-edge-cloud hierarchical federated learning," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2020, pp. 1–6.

[23] C. Xu, Y. Qu, Y. Xiang, and L. Gao, "Asynchronous federated learning on heterogeneous devices: A survey," *Comput. Sci. Rev.*, vol. 50, Nov. 2023, Art. no. 100595.

[24] X. Yu et al., "Async-HFL: Efficient and robust asynchronous federated learning in hierarchical IoT networks," in *Proc. 8th ACM/IEEE Conf. Internet Things Design Implement.*, May 2023, pp. 236–248.

[25] W. Wu, L. He, W. Lin, R. Mao, C. Maple, and S. Jarvis, "SAFA: A semi-asynchronous protocol for fast federated learning with low overhead," *IEEE Trans. Comput.*, vol. 70, no. 5, pp. 655–668, May 2021.

[26] M. Chen, B. Mao, and T. Ma, "FedSA: A staleness-aware asynchronous federated learning algorithm with non-IID data," *Future Gener. Comput. Syst.*, vol. 120, pp. 1–12, Jul. 2021.

[27] C. Briggs, Z. Fan, and P. Andras, "Federated learning with hierarchical clustering of local updates to improve training on non-IID data," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2020, pp. 1–9.


Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.

[28] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," 2018, *arXiv:1806.00582*.

[29] M. Seol and T. Kim, "Performance enhancement in federated learning by reducing class imbalance of non-IID data," *Sensors*, vol. 23, no. 3, p. 1152, Jan. 2023.

[30] G. Damaskinos, R. Guerraoui, A.-M. Kermarrec, V. Nitu, R. Patra, and F. Taiani, "FLeet: Online federated learning via staleness awareness and performance prediction," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 5, pp. 1–30, Oct. 2022.

[31] C. Zhao et al., "Temporal spectrum cartography in low-altitude economy networks: A generative AI framework with multi-agent learning," 2025, *arXiv:2505.15571*.

[32] C. Zhao et al., "Generative AI-enabled wireless communications for robust low-altitude economy networking," 2025, *arXiv:2502.18118*.

[33] Y. Sun, L. Zhang, L. Guo, J. Li, D. Niyato, and Y. Fang, "S-RAN: Semantic-aware radio access networks," *IEEE Commun. Mag.*, vol. 63, no. 4, pp. 207–213, Apr. 2025.

[34] S. Hua, Y. Sun, K. Ma, D. Niyato, and M. Ali Imran, "A mathematical framework of semantic communication based on category theory," 2025, *arXiv:2504.11334*.

[35] W. Yuan et al., "From ground to sky: Architectures, applications, and challenges shaping low-altitude wireless networks," 2025, *arXiv:2506.12308*.

[36] J. Wang et al., "Generative AI based secure wireless sensing for ISAC networks," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 5195–5210, 2025.

[37] G. Sun et al., "Aerial reliable collaborative communications for terrestrial mobile users via evolutionary multi-objective deep reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 24, no. 7, pp. 5731–5748, Jul. 2025.

[38] X. Ren, H. Liang, Y. Wang, C. Zhang, Z. Xiong, and L. Zhu, "BESA: Boosting encoder stealing attack with perturbation recovery," 2025, *arXiv:2506.04556*.

[39] C. Xie et al., "Disrupting vision-language model-driven navigation services via adversarial object fusion," 2025, *arXiv:2505.23266*.

[40] H. Zhang and L. Hanzo, "Federated learning assisted multi-UAV networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 11, pp. 14104–14109, Nov. 2020.

[41] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Dec. 2017, pp. 1707–1718.

[42] Q.-V. Pham, M. Zeng, R. Ruby, T. Huynh-The, and W.-J. Hwang, "UAV communications for sustainable federated learning," *IEEE Trans. Veh. Technol.*, vol. 70, no. 4, pp. 3944–3948, Apr. 2021.

[43] X. Hou, J. Wang, C. Jiang, X. Zhang, Y. Ren, and M. Debbah, "UAV-enabled covert federated learning," *IEEE Trans. Wireless Commun.*, vol. 22, no. 10, pp. 6793–6809, Oct. 2023.

[44] T. Zeng, O. Semiari, M. Mozaffari, M. Chen, W. Saad, and M. Bennis, "Federated learning in the sky: Joint power allocation and scheduling with UAV swarms," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2020, pp. 1–6.

[45] Q.-V. Pham, M. Le, T. Huynh-The, Z. Han, and W.-J. Hwang, "Energy-efficient federated learning over UAV-enabled wireless powered communications," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 4977–4990, May 2022.

[46] H. Yang, J. Zhao, Z. Xiong, K.-Y. Lam, S. Sun, and L. Xiao, "Privacy-preserving federated learning for UAV-enabled networks: Learning-based joint scheduling and resource management," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 10, pp. 3144–3159, Oct. 2021.

[47] M. Fu, Y. Shi, and Y. Zhou, "Federated learning via unmanned aerial vehicle," *IEEE Trans. Wireless Commun.*, vol. 23, no. 4, pp. 2884–2900, Apr. 2024.

[48] X. Lu, Y. Liao, P. Lio, and P. Hui, "Privacy-preserving asynchronous federated learning mechanism for edge network computing," *IEEE Access*, vol. 8, pp. 48970–48981, 2020.

[49] J. Nguyen et al., "Federated learning with buffered asynchronous aggregation," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2021, pp. 3581–3607.

[50] T. Zhang, A. Song, X. Dong, Y. Shen, and J. Ma, "Privacy-preserving asynchronous grouped federated learning for IoT," *IEEE Internet Things J.*, vol. 9, no. 7, pp. 5511–5523, Apr. 2022.

[51] M. Dai, Y. Zhao, J. Yuan, S. Kianoush, S. Savazzi, and B. Li, "Federated learning based on asynchronous and adjusted client training," *Phys. Commun.*, vol. 61, Dec. 2023, Art. no. 102164.

[52] R. Zhang, W. Luo, Y. Luo, H. Zhang, and J. Wang, "AFL-DCS: An asynchronous federated learning framework with dynamic client scheduling," *Eng. Appl. Artif. Intell.*, vol. 133, Jul. 2024, Art. no. 107927.

[53] J. O. Odeh, X. Yang, C. I. Nwakanma, and S. Dhelim, "Asynchronous privacy-preservation federated learning method for mobile edge network in industrial Internet of Things ecosystem," *Electronics*, vol. 13, no. 9, p. 1610, Apr. 2024.

[54] Z. Zhai, X. Yuan, X. Wang, and H. Yang, "UAV-enabled asynchronous federated learning," *IEEE Trans. Wireless Commun.*, vol. 24, no. 3, pp. 2358–2372, Mar. 2025.

[55] Z. Cui, T. Yang, X. Wu, H. Feng, and B. Hu, "The data value based asynchronous federated learning for UAV swarm under unstable communication scenarios," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 7165–7179, Jun. 2024.

[56] M. S. H. Abad, E. Ozfatura, D. Gunduz, and O. Ercetin, "Hierarchical federated learning ACROSS heterogeneous cellular networks," in *Proc. IEEE Int. Conf. ICASSP*, Apr. 2020, pp. 8866–8870.

[57] T. Wang, X. Huang, Y. Wu, L. Qian, B. Lin, and Z. Su, "UAV swarm-assisted two-tier hierarchical federated learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 1, pp. 943–956, Sep. 2023.

[58] S. Wang, S. Hosseinalipour, M. Gorlatova, C. G. Brinton, and M. Chiang, "UAV-assisted online machine learning over multi-tiered networks: A hierarchical nested personalized federated learning approach," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 2, pp. 1847–1865, Jun. 2023.

[59] R. Khelf, E. Driouch, and W. Ajib, "On the optimization of UAV-assisted wireless networks for hierarchical federated learning," in *Proc. IEEE 34th Annu. Int. Symp. Pers., Indoor Mobile Radio Commun. (PIMRC)*, Sep. 2023, pp. 1–6.

[60] F. Zhao, B. Yang, C. Li, C. Zhang, L. Zhu, and G. Liang, "Two-layer consensus based on primary-secondary consortium chain data sharing for Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 73, no. 9, pp. 13828–13838, Sep. 2024.

[61] Q. Miao, H. Lin, X. Wang, and M. M. Hassan, "Federated deep reinforcement learning based secure data sharing for Internet of Things," *Comput. Netw.*, vol. 197, Oct. 2021, Art. no. 108327.

[62] Z. Yang, X. Zhang, D. Wu, R. Wang, P. Zhang, and Y. Wu, "Efficient asynchronous federated learning research in the Internet of Vehicles," *IEEE Internet Things J.*, vol. 10, no. 9, pp. 7737–7748, May 2023.

[63] W. H. Huff et al., "DHA-FL: Enabling efficient and effective aiot via decentralized hierarchical asynchronous federated learning," in *Proc. MLSys Workshop Resource-Constrained Learn. Wireless Netw.*, 2023.

[64] N. Su and B. Li, "Asynchronous federated unlearning," in *Proc. IEEE Conf. Comput. Commun.*, May 2023, pp. 1–10.

[65] J. Yang, Y. Zhou, W. Wen, J. Zhou, and Q. Zhang, "Asynchronous hierarchical federated learning based on bandwidth allocation and client scheduling," *Appl. Sci.*, vol. 13, no. 20, p. 11134, Oct. 2023.

[66] N. Wang, X. Zhao, Y. Jiang, and Y. Gao, "Iterative metric learning for imbalance data classification," in *Proc. Twenty-Seventh Int. Joint Conf. Artif. Intell.*, Jul. 2018, pp. 2805–2811.

[67] N. Wang, R. Liang, X. Zhao, and Y. Gao, "Cost-sensitive hypergraph learning with F-measure optimization," *IEEE Trans. Cybern.*, vol. 53, no. 5, pp. 2767–2778, May 2023.

[68] F. Xu, N. Wang, H. Wu, X. Wen, X. Zhao, and H. Wan, "Revisiting graph-based fraud detection in sight of heterophily and spectrum," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 8, pp. 9214–9222.

[69] T. Zhang, R. Ramakrishnan, and M. Livny, "BIRCH: An efficient data clustering method for very large databases," *ACM SIGMOD Rec.*, vol. 25, no. 2, pp. 103–114, Jun. 1996.

[70] C. Xing Tian, Y. Liu, H. Li, R. C. C. Cheung, and S. Wang, "Gradient-congruity guided federated sparse training," 2024, *arXiv:2405.01189*.

[71] Y. Chen, Y. Ning, M. Slawski, and H. Rangwala, "Asynchronous online federated learning for edge devices with non-IID data," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2020, pp. 15–24.


Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.



**Xiangyun Tang** (Member, IEEE) received the B.Eng. degree in computer science from the Minzu University of China, Beijing, China, in 2016, and the Ph.D. degree in cyberspace security from Beijing Institute of Technology, Beijing, in 2022. She is currently an Associate Professor with the School of Information Engineering, Minzu University of China. She has published more than 30 papers, including AAAI, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE NETWORK, IEEE WIRELESS COMMUNICATIONS LETTERS, IEEE GLOBECOM, IEEE ICC, and IEEE WCNC. Her research interests include secure multi-party computation and artificial intelligence security. She was a recipient of the Best Paper Award from IEEE ICBCTIS 2023 and ICA3PP 2024 and the Outstanding Paper Award from IEEE iThings 2023. She served as a guest editor for multiple journals and the TPC chair and a PC member for international conferences and workshops.



**Ruiqi Peng** received the B.Eng. degree in computer science and technology from Jishou University, Jishou, China, in 2024. He is currently pursuing the degree with the School of Information Engineering, Minzu University of China. His research interests include federated learning and federated unlearning.



**Hao Li** received the B.Eng. degree in software engineering from Hebei Normal University, Shijiazhuang, China, in 2024. She is currently pursuing the degree with the School of Information Engineering, Minzu University of China. Her research interests include federated learning and federated unlearning.



**Tao Zhang** received the dual B.S. degree in Internet of Things engineering from Beijing University of Posts and Telecommunications (BUPT) and the Queen Mary University of London in 2018 and the Ph.D. degree in computer science and technology from BUPT in 2023. He is currently an Associate Professor with the School of Cyberspace Science and Technology, Beijing Jiaotong University. His publications include ESI highly cited article and well-archived international journals and proceedings, such as IEEE COMMUNICATIONS SURVEYS AND TUTORIALS, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING, and IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS. His research interests include network security, moving target defense, and federated learning. He served as a Guest Editor for *Electronics* and *Chinese Journal of Network and Information Security* and the TPC chair and a PC member for some international conferences and workshops. He was a recipient of the Best Paper Award from NaNA 2018, IWCMC 2021, DIONE 2024, and ICA3PP 2024, and the Outstanding Paper Award from iThings 2023 and SmartCity 2024. His Ph.D. thesis was awarded the Outstanding Doctoral Dissertation by BUPT in 2023.



**Xiangwang Hou** (Member, IEEE) received the Ph.D. degree in electronic engineering from Tsinghua University, Beijing, China, in 2025. He is currently a Post-Doctoral Researcher (Shuimu Tsinghua Scholar Program) with the Department of Electronic Engineering, Tsinghua University. From 2023 to 2024, he was a Joint Ph.D. Student at the School of Computer Science and Engineering, Nanyang Technological University, Singapore, under the supervision of Prof. Dusit Niyato. From 2020 to 2021, he worked as an Algorithm Engineer at Huawei Technologies Company Ltd., Tsinghua University. His research interests include edge intelligence, federated learning, wireless AI, and UAV/AUV networks. He was a recipient of the Best Paper Award at IEEE ICC and WCSP, and he has also served as a Guest Editor for IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING.



**Xiangzhi Liu** received the M.S. and Ph.D. degrees from Shandong University, Jinan, China. He is currently a Professor with the Qilu University of Technology (Shandong Academy of Sciences), Jinan. His research interests include intelligence, generative AI, and information security.

Authorized licensed use limited to: Anhui University. Downloaded on August 02,2026 at 07:28:20 UTC from IEEE Xplore. Restrictions apply.